Agile

Agile methodologies have evolved since the 1980s, however it didn't become a popular choice as an approach to product development until the 21st Century.

There are many approaches that precede Agile, but focused on some of the aspects that Agile encompasses, such as iterative development using Boehm spiral model, or approaches that aimed to deliver value quickly such as Rapid Application Development and Feature Driven Development. Others focused more on continuous improvement, such as Lean Thinking. The Agile Alliance formed in 2001, at which point the Agile Manifesto was created and signed and the 12 principles were documented.

Agile as a concept has evolved, and will continue to do so, because one of its premises is continuous improvement.

There are many different flavours of Agile, offering different methodologies, this pack includes a generic overview, with some focus on Scrum and Kanban.

The Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work, we have come to value:

Individuals and interactions over processes and tools.

Working software over comprehensive documentation.

Customer collaboration over contract negotiation.

Responding to change over following a plan.

That is, while there is value on the items on the right, we value the items on the left more.

12 Principles Underpinning the Manifesto

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Business people and developers must work together daily throughout the project.

Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Working software is the primary measure of progress.

Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

Continuous attention to technical excellence and good design enhances agility.

Simplicity–the art of maximizing the amount of work not done–is essential.

The best architectures, requirements, and designs emerge from self-organizing teams.

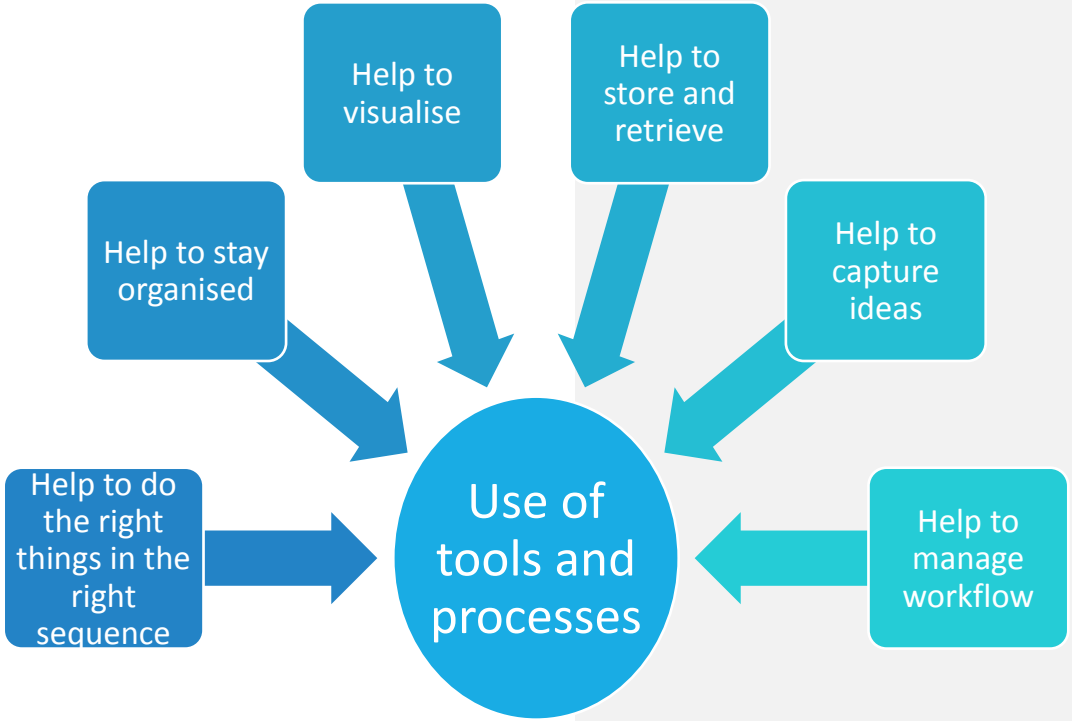
At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

Individuals and Their Interactions Over Processes and Tools

We shall now look at the four elements of the manifesto in more detail.

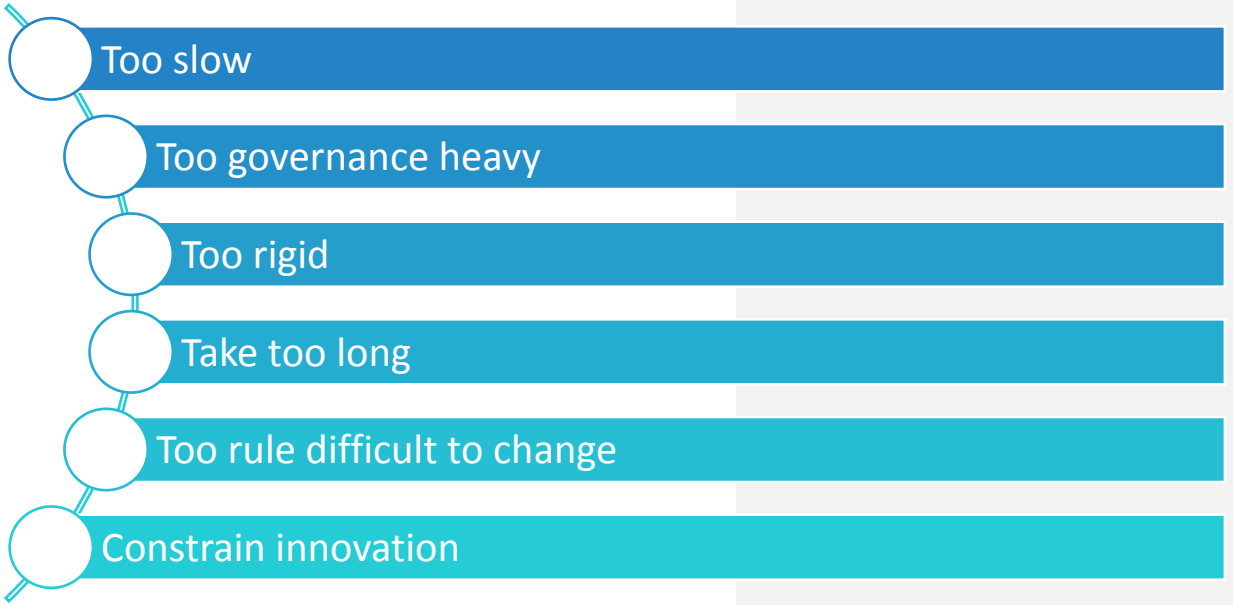
Individuals and their Interactions over Processes and Tools

As with all methodologies, processes and tools are available to assist various activities, such as analysis, ideating and tracking, however, it is the interaction between team members and interactions with the customer that are more meaningful. The processes and tools should enable the interactions.



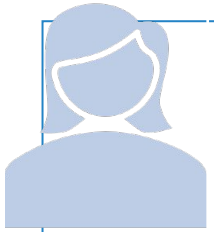
Individuals and Their Interactions Over Processes and Tools

Potential downsides



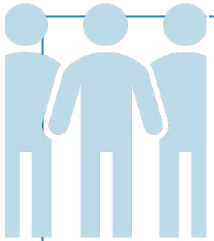
Agile teams

Role names will vary across different organisations and may depend on the flavour of Agile methodology adopted as a standard.



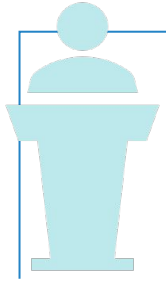
Product Owner/Manager

- Drives Project success
- Represents the beneficiary
- Decision maker
- Manages Backlog



Development Team

- Multiple skills within team including:
- - Business Analyst
- - Designer/Architect
- - Developer/Coder
- -Testers



Scrum Master

- Ensures work is understood and done
- Facilitates other roles
- Removes impediments

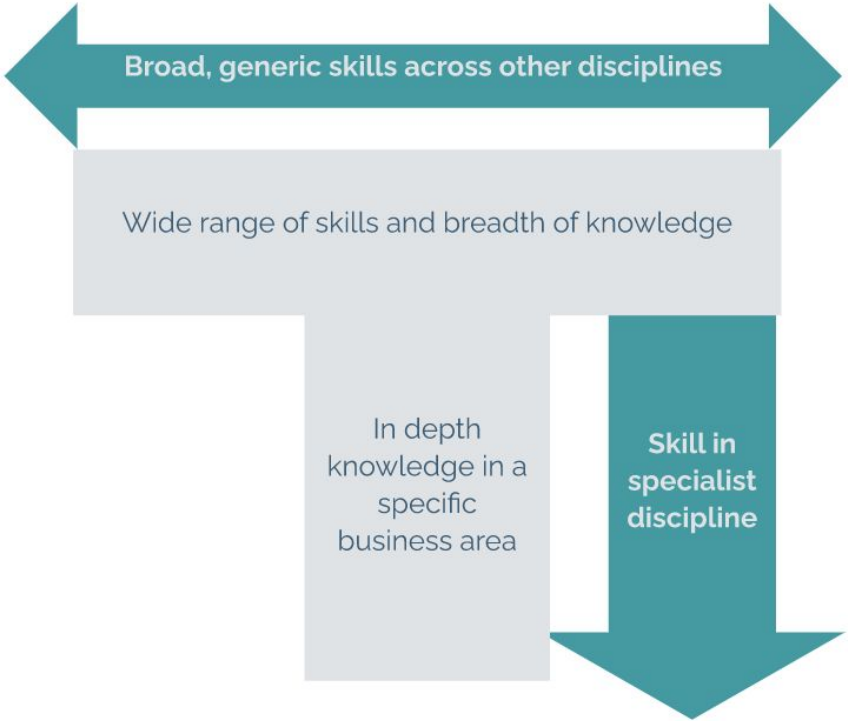
Agile teams


Agile lead
• Guides team and creates Agile culture
• Facilitates, coaches, removes impediments
Stakeholder role
• Those impacted by or interested in the change
• Provide subject matter expertise
• May influence decisions
Customer
• Internal or external beneficiaries
• Provide and sign off stories
• Feed back

Agile Team members – T-shaped Professionals

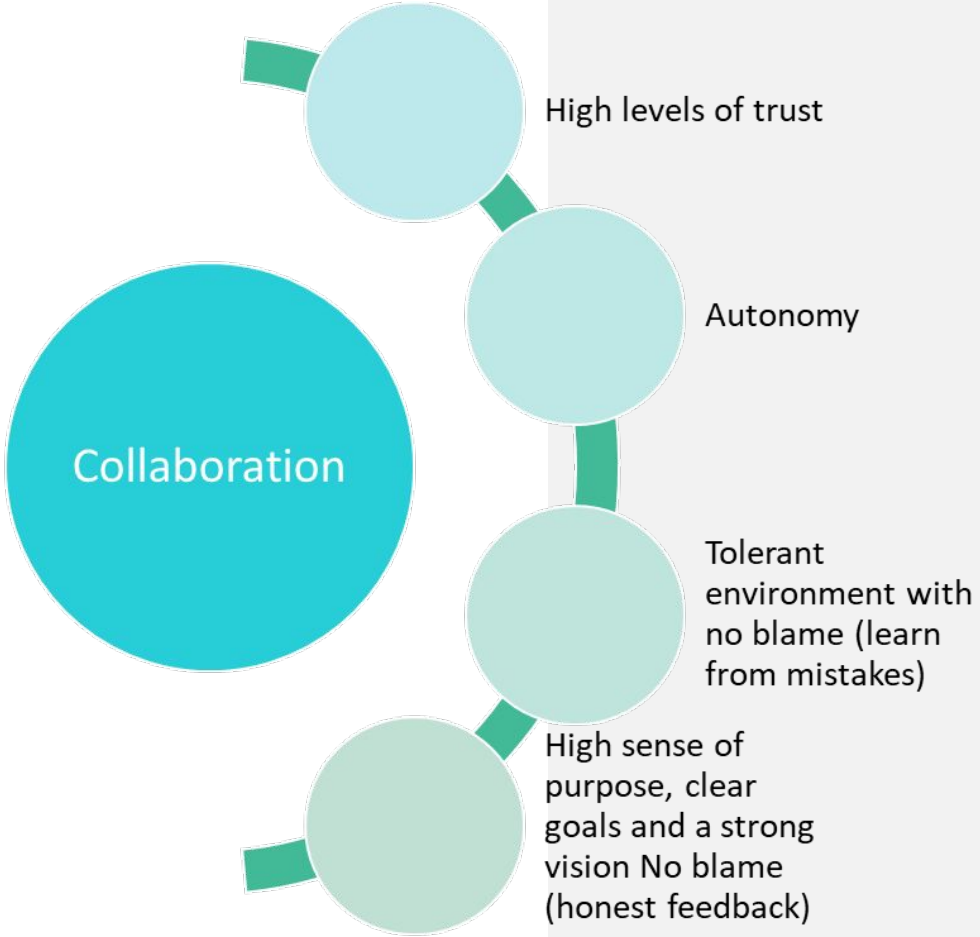
Ideally, the members of Agile teams should T-shaped professionals. This means that they have a broad range of generic skills across all related disciplines as well as an in depth knowledge in their specialism.
For example, a tester should have an in depth knowledge of testing tools and building test cases, but ideally will also have a broad understanding of different aspects of development and integration as well as understanding the business domain.

The T shaped professional.



Collaboration

Agile teams need to work closely together and communicate frequently, to discuss the best way to build and deliver each feature and ensure that the business needs will be met.



Collaboration

Team members need to do the following in order that the team work effectively.

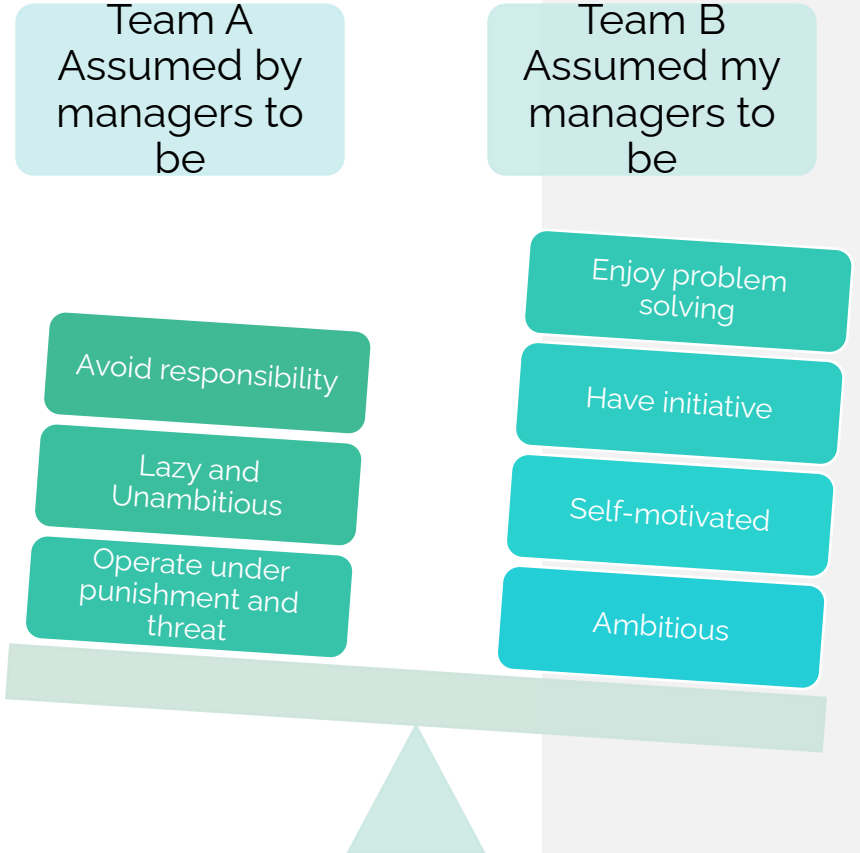
- Value each others skills
- Share knowledge
- Help each other
- Honest feedback
- Work together to achieve goals
- Mutual respect
- No individual glory

Self-organising Autonomous Teams

- Self motivated
- Self organised
- Empowered to decide
- Troubleshoot issues

Self-organising Autonomous Teams

Which team is likely to perform well?



McGregor and Gershenfeld – The Human Side of Enterprise

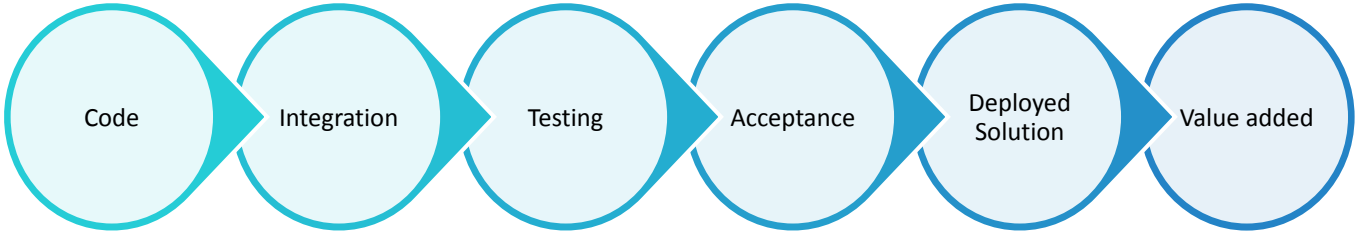
Work is carried out by Agile teams, there may be one or more teams working towards the same set of product improvements.

Studies have found that that team motivation, productivity and innovation is vastly improved if the teams are set clear goals and then left to be autonomous. They work best in a task-based culture where the achievement of the teams goals is of utmost priority. Individual glory and bureaucracy are not features of this environment.

The Value of Working Software

Working software means a working solution which will contain both IT and Non IT elements. Every delivery must add value and this cannot be achieved through code drops alone.

- Demonstrating progress through constant delivery



The Value of Working Software

- Being able to deliver high value software frequently

People

Training and comms

Process in place

Structural changes

Documentation for operational use and maintenance

The Value of Working Software

The full working solution can only be evidenced when it is in the live environment, however the product owner, customers and other stakeholders may need to approve the software for release in a simulated environment. Prototypes are commonly used to demonstrate working software. It should be noted that some prototypes are purely a visual representation. Others have working functionality and may be based on realistic business scenarios, but there are likely to be aspects of the environment that are not well simulated and where possible, the teams need to ensure that they have a robust release plan in order that the solution can be checked over in the live environment and backed out if it is not operating as expected.

The Importance of Documentation

- User documentation
- Sufficient documentation to maintain and support the software
- Other documentation that is fit for purpose, for example the product backlog

Keep things that add value, eliminate things that don’t add value. Agile teams need to consider each potential piece of documentation to check whether it adds value for product development or future maintenance.

The Seven Wastes of Lean Software Development

Eliminating waste is fundamental to lean. Waste can be found in many aspects of work, for example wasting time walking to the back of the factory to pick up tools several times a day when the tools could instead be moved to the front of the factory floor.

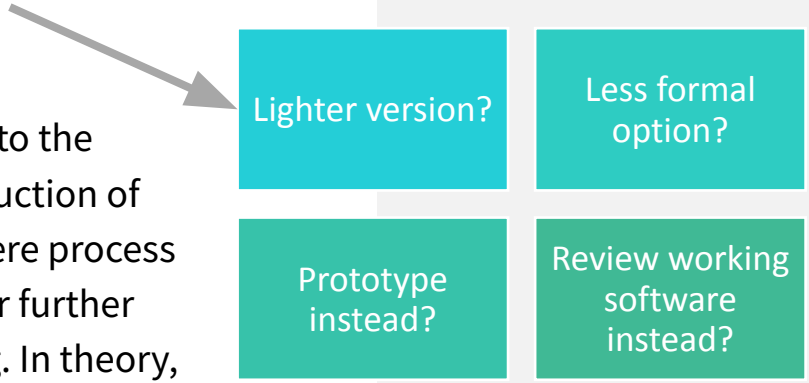
1. Partially done work
2. Extra Processes
3. Extra Features
4. Task Switching
5. Waiting
6. Motion
7. Defects

The Seven Wastes of Lean Software Development

The element of the Agile manifesto, working software over comprehensive documentation is related to the 2nd of the lean wastes, over processing. The principle is that sufficient documentation should be produced (to support the change and to support the software in the future), anything over and above that would be a waste, the documentation would be over engineered. Working software on the other hand should add value to the solution and not wasteful. Note however that it is theoretically possible to build working software that does not add value, therefore, in Agile the highest priority features are built first and one of the key aspects to consider when judging priority is the value add.

The Seven Wastes of Lean Software Development

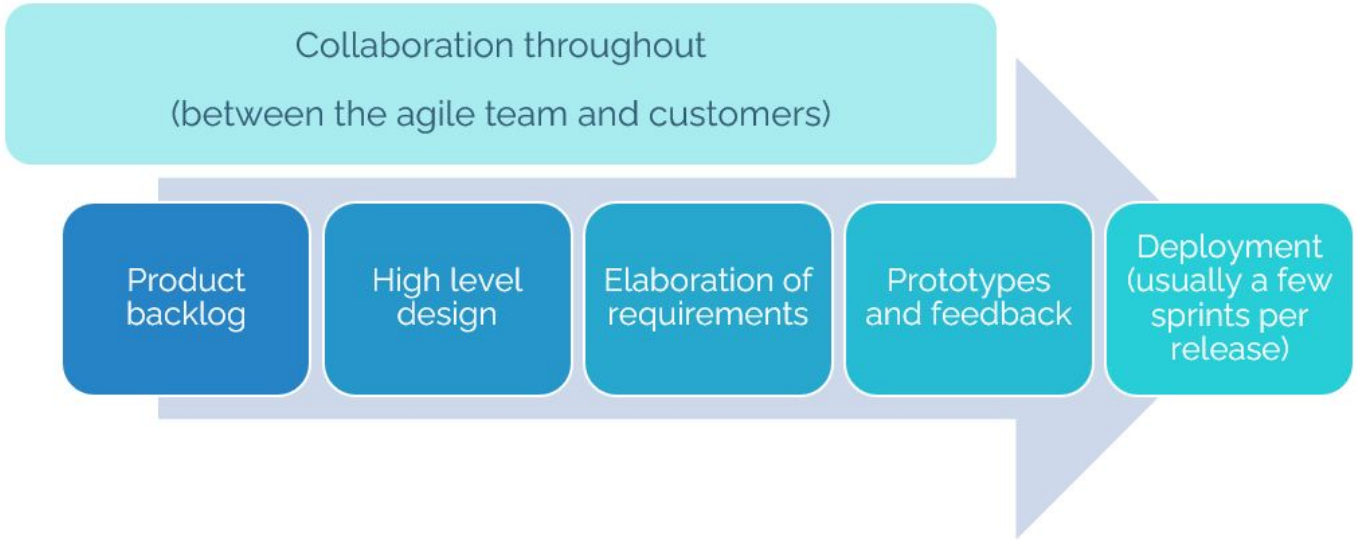
If it is needed, how comprehensive does documentation need to be? Does it need to specify in detail, or give a high level view?



The wastes of lean not only apply to documentation, but to the software and non-software elements of the solution. Reduction of wastes can continue into business-as-usual activities where process can be continuously made. This may result in the need for further software changes, and these can be added to the backlog. In theory, Agile development doesn't have the time constraints of a traditional project, because the focus is on continuous improvement, product development and innovation, rather than change projects. In practice however, many organisations do not move away from the traditional iron triangle project constraints because the cultural shift is too great.

Customer Collaboration over Contract Negotiation

The Agile team's relationship with its customers



Customer Collaboration over Contract Negotiation

Customers will be using the system and will know what goals and outcomes they need to achieve when using it. They input user stories into the initial backlog and need to be involved in further stages of elaboration as well as providing feedback when outputs are produced.

Customers have different perspectives

Product owners represent the customer viewpoint

Customers have different goals

Customers have different areas of knowledge

Customer Collaboration over Contract Negotiation

The first rule is to know who your customers are. Sometimes it is worth drawing a Use Case Diagram to show who will interact with the system. This will help to identify internal users of the system, including managers and recipients of reports, as well as any end users, maybe customers, maybe suppliers. You may work for a team that produces software for another company. Perhaps you work for a software house or another third-party supplier. In this case you will see the people who are paying for the software as customers, but you also need to see their internal users and their customers as customers of the product.

After drawing up a Use Case diagram, consider conducting user role analysis and maybe even creating personas. The product owner represents the customer view, but if you have access to actual customers or can create personas for them. It is easier to provide features that will provide value to them.

Personas

Personas are fictional characters used to represent different user types. They are based on research, or past experience. They help teams to empathise with users and to consider how users will engage with the product.

Learn more about personas here.